

A DataOps and MLOps Architecture for Real-World and Real-Time Assessment of Functional Knee Instability Using IMU Sensors

Michaël Johan Lodewijk Groenewegen van der Weijden¹, Georgiana Manolache¹,
Nico Kuijpers¹, Petra Heck¹, and Jacco Snoeren¹

¹Master of Applied IT, Fontys University of Applied Sciences

Abstract

Anterior cruciate ligament (ACL) rupture often leads to functional knee instability that is difficult to quantify in daily life and sports settings. Existing assessments rely on static, clinician-performed tests and intermittent imaging, which provide limited insight into functional knee instability under load. This study proposes an architecture, grounded in DataOps & MLOps principles, for the real-time collection and processing of inertial measurement unit (IMU) data to assess functional knee instability in real world settings. The architecture combines a Kubernetes-based microservices platform, a Kafka-centered event backbone, and an asset-oriented data orchestration layer to support scalable ingestion, validation, feature extraction, model training, and online inference. DataOps & MLOps architectural principles were elicited from literature, project documentation, and stakeholder workshops, and were formalized across four domains: 1) Data Management, 2) ML Lifecycle, 3) Infrastructure, and 4) Quality Attributes. A proof-of-concept implementation demonstrates that the platform can stream IMU data from wearable sensors, store and version data and models, and expose real-time features and predictions via dashboards for clinicians. The proposed architecture provides a reusable, extensible foundation for standardized, interpretable metrics of functional knee instability and can be extended in future work with clinical validation and advanced analytics.

1 Introduction

Physical activity is central to personal identity, and restrictions affect both mental and physical health [49]. Everyday actions such as walking require little conscious effort because the knee's ligaments and muscles coordinate to maintain stability. The anterior cruciate ligament (ACL) is a primary stabilizer of the knee, and after rupture many patients perceive the knee as lax or unstable and describe episodes of "giving way". At present, clinicians can monitor the consequences of an ACL rupture only intermittently in the clinic and not continuously in everyday life.

In current practice, clinicians diagnose ACL rupture using history, physical examination, and magnetic resonance imaging (MRI) [28]. Anterior knee laxity is commonly assessed with the Lachman and pivot-shift tests [28]. These evaluations are largely static and partly subjective, and they provide limited insight into dynamic knee stability during sport-specific tasks or the precise timing of episodes of instability.

Despite accurate devices for specific aspects of laxity, a recent review concluded that there is still no agreed objective measure of knee laxity suitable for routine clinical use, and that wearable technologies remain underutilized in standard pathways [5].

The lack of objective measures of dynamic instability can delay treatment, weaken confidence in treatment selection, and prolong rehabilitation. ACL rupture prevents an estimated 20 to 60 percent of athletes from returning to their preinjury sport level [20]. At the same time, ACL injuries are common: in the Netherlands, about 3 percent of amateur athletes and more than 15 percent of professional athletes sustain an ACL tear each year, and more than 9,000 ACL reconstructions are performed annually [20].

Without agreed thresholds or labels, medical professionals as well as (supervised) machine learning models risk making subjective judgments, and static metrics may miss episodes that occur only under load. This uncertainty contributes to variation in clinical decision making and makes it difficult to compare outcomes across centers and patient groups [5]. Within this context, (supervised) machine learning models process raw sensor signals, preprocessed inputs, or engineered features and function as clinical interpreters that transform IMU data into an assessment of whether an instability episode has occurred.

Wearable inertial measurement units (IMUs) capture acceleration and angular velocity during functional tasks, have been validated against optical reference systems, and can approximate key load surrogates [2]. When wearable devices stream high-frequency IMU signals to a governed data platform with low latency, clinicians can observe changes in mobility, balance, and activity patterns, such as instability during pivoting tasks, as they occur. This makes it possible to intervene before these changes become clinically apparent.

ACL-specific applications of IMU-based kinetics have begun to emerge. A recent systematic review on estimating kinetics from IMUs for ACL rehabilitation concluded that IMU-based models can approximate joint loads and GRFs during rehabilitation exercises, but often rely on multiple sensors and complex modelling, and are rarely validated on ACL-injured populations outside of laboratory environments [38]. The review emphasized the need for standardized pipelines, and systems that can operate in clinics and field settings.

At the level of concrete systems, a low-cost setup using two shank-mounted IMUs can estimate spatiotemporal gait parameters such as stride time, stance time, cadence, and walking speed. The design illustrates that basic gait analytics can be implemented with hardware and smartphone-based data logging, but processing is performed offline and the system is not integrated into a governed data platform [32].

This study makes three primary contributions. First, it defines a reusable reference architecture that integrates DataOps and MLOps principles for the real-time processing of IMU data in the context of functional knee instability. Second, it operationalizes abstract DataOps and MLOps concepts into concrete architectural design principles spanning 1) Data Management, 2) ML Lifecycle, 3) Infrastructure, and 4) Quality Attributes [40, 44, 33, 54, 45, 46], thereby bridging the gap between conceptual guidelines and deployable systems. Third, it demonstrates the technical feasibility of the proposed architecture through a proof-of-concept implementation that streams wearable sensor data, manages data and model versioning, and exposes clinically interpretable outputs to support decision making.

The remainder of this paper is organized as follows. Section 2 describes the methodological approach adopted in this study. Section 3 presents the results of the proposed architecture and its functionality. Section 4 discusses these findings and their implications. Finally, Section 5 concludes the paper and summarizes the main findings.

2 Methodology

An organized, multi-method research approach was used to inform the design. First, academic literature, industry reports, and technical documentation were systematically reviewed to clarify the problem domain, identify relevant design patterns, and map out existing solutions and their limitations. Second, product and document analysis of current systems in the knee instability project provided insight into practical constraints, and typical data flows (Appendix A).

Expert input was gathered through stakeholder meetings and review sessions with domain experts and software architects. These methods were employed to investigate the challenges of collecting, processing, and managing real-time IMU data and to ground architectural decisions in both theory and practice.

Collectively, these inputs were used to formulate architectural principles. The resulting architectural principles (Appendix B) guided the selection of technologies and the definition of the data and model pipelines. The methodology combines desk research, qualitative stakeholder engagement, and design-oriented analysis to propose an architecture that can be evaluated in future experimental and clinical studies.

2.1 Requirements

Over the course of the project, weekly stakeholder meetings were held. These meetings were used to elicit and refine requirements, share perspectives, and align project objectives. Participants included software students, software engineers, and a domain expert responsible for connecting the technical work to clinical needs.

All research findings and design decisions were reviewed with stakeholders. The architecture principles (Appendix B) were evaluated by experienced software engineers and architects, and the data-processing procedures within the architecture were validated by medical professionals experienced in knee instability. These reviews were instrumental in keeping the research clinically relevant and in validating the emerging requirements and design.

2.2 Architectural Design Principles

Company documents, peer-reviewed literature, and technical blogs were systematically reviewed (Appendix A), and stakeholder input was incorporated to clarify operational constraints. This analysis yielded a set of architectural design principles to guide subsequent decisions formalized across four domains: 1) Data Management, 2) ML Lifecycle, 3) Infrastructure, and 4) Quality Attributes. These architectural design principles were operationalized into explicit selection and evaluation criteria and used to assess candidate tools, platforms, and services, as well as the proposed reference architecture, with respect to strategic fit, technical feasibility, operational impact, and stakeholder priority utilizing the MoSCoW method, which classifies each item as a “must have,” “should have,” “could have,” or “won’t have” in the current iteration [15].

To demonstrate that the proposed architecture is capable of ingesting and processing real-time IMU sensor data, its validation combines structured stakeholder discussions, domain expert review, systematic tracing against the DataOps & MLOps architectural design principles (Appendix B), and a technical proof of concept. Selected components and technologies are referenced against concrete architectural design principles, ensuring traceability from requirements to design choices and providing evidence that the proposed architecture can fulfill the requirements. Stakeholder workshops and interviews were used to assess the practical relevance and acceptability of the architecture in clinical and organizational contexts, while domain experts review the design to judge its technical capability. In addition, a technical proof-of-concept implementation was developed to demonstrate that the proposed architecture can be translated into a working system in a realistic setting. Together, these activities provide converging evidence that the selected technologies and architectural principles

are both requirement-compliant and suitable for real-world adoption, thereby informing the final architectural design and implementation.

3 Results

3.1 Architectural Design Principles

DataOps & MLOps architectural design principles were formalized across four domains that mirror the main architectural concerns of the platform: 1) Data Management, 2) ML Lifecycle, 3) Infrastructure, and 4) Quality Attributes. Within each domain, the architectural design principles were prioritized with the MoSCoW method in collaboration with stakeholders and were used as requirements for the design of the final architecture [15]. The detailed requirement tables in Appendix B list these items and use unique identifiers (e.g. AP-DM for Data Management, AP-ML for the ML Lifecycle, AP-I for Infrastructure and AP-QA for Quality Attributes), which are referenced throughout section 3.2.

3.2 Architecture

This section outlines the structure and components of the proposed architecture (Figure 1) utilizing C4 Level 2 [34] demonstrating how the architectural design principles have been applied to move to a real-world setting, real-time IMU sensor data handling.

The architecture is designed combining microservice and messaging design principles [14]. The platform consists of a collection of small, independent services that communicate by exchanging messages. This design approach yields a platform that is highly 1) scalable and performant, 2) flexible and extensible, and 3) resilient and robust to failures.

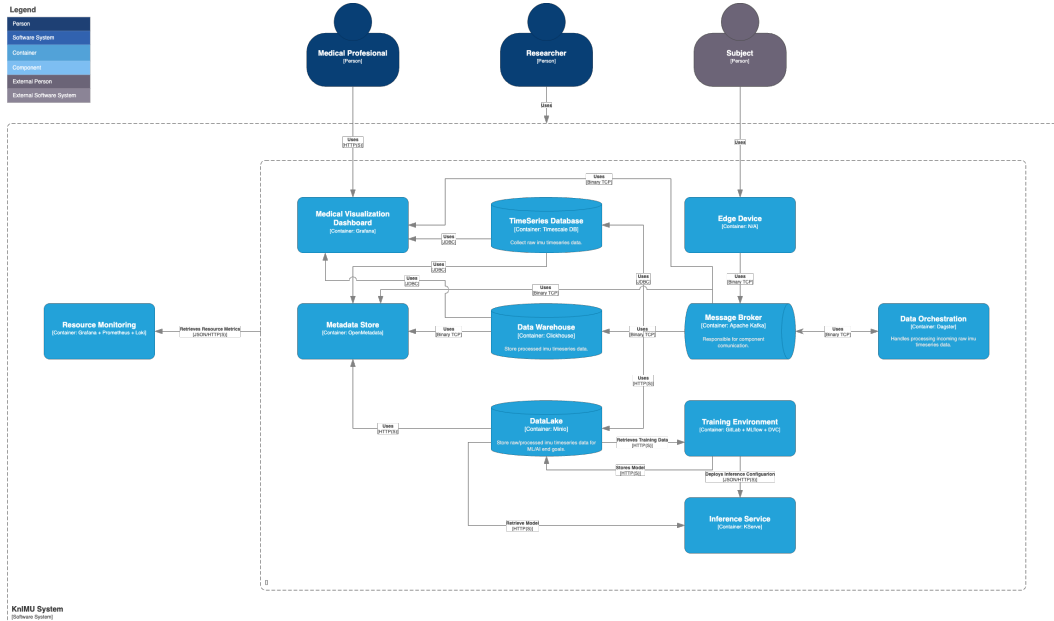


Figure 1: C4 Level 2 Architecture Diagram depicting an overview of the proposed DataOps & MLOps driven architecture. The platform integrates microservices, event streaming, orchestration, storage, model training, inference, monitoring, and metadata management to enable scalable, real-time ingestion and processing of IMU data for clinical decision support in functional knee instability.

Microservices. Kubernetes orchestrates loosely coupled, scalable microservices that support data-centric operations capable of meeting complex data-management demands, such as handling large data volumes and high-velocity streams while preserving data integrity for reliable analysis (AP-DM-8). Containerization and Infrastructure as Code (IaC) reinforce these properties by ensuring that identical container images, when executed with the same inputs, consistently produce the same outputs, thereby providing strong reproducibility guarantees across data-processing components (AP-I-9, AP-QA-0) [1].

Building on this foundation, the Kubernetes-based microservices architecture fulfills key scalability and resilience requirements. Kubernetes can replicate services and shard data across nodes, enabling horizontal scaling and fault tolerance (AP-I-5) [8]. Its service-discovery mechanisms allow new microservice instances, and even entirely new services or data sources, to join or leave the system dynamically without manual reconfiguration. As a result, the architecture can evolve and ingest new data sources at runtime without downtime (AP-I-7). Moreover, Kubernetes adjusts compute and storage allocations dynamically to match workload demands while minimizing resource waste, including selectively scaling edge and central components as conditions change (AP-I-0, AP-I-4, AP-I-8) [3, 17].

To operate this architecture reliably in practice, the microservices stack incorporates DevOps and monitoring best practices for both data and machine learning components. The same IaC and containerization mechanisms underpin consistent, reproducible environments for data processing and model serving, while continuous integration and deployment pipelines automatically build, test, and release new service versions when changes are committed (AP-QA-0) [3].

Message Broker. Apache Kafka serves as the platform’s central event backbone for data management and lineage. In combination with Kafka Schema Registry, it requires producers and consumers to use a shared, versioned schema, enforcing a consistent data model across components and preventing incompatible payloads from entering the streams (AP-DM-1, AP-DM-8) [52]. Through Kafka Connect and custom producers, topics ingest data from heterogeneous sources (AP-DM-5, AP-DM-8), while events are persisted in an append-only, partitioned log with configurable retention (AP-DM-3, AP-DM-6) [9].

Kafka provides a resilient, horizontally scalable, and loosely coupled runtime. Topics are partitioned and replicated across brokers, and leader-follower replication with automatic failover enables the cluster to tolerate broker failures while maintaining availability and durability (AP-I-5, AP-I-7) [9]. Its publish/subscribe model ensures that communication between data producers and consumers is asynchronous and decoupled (AP-DM-2, AP-I-6) [9]. Furthermore, Kafka can be deployed on edge or other resource-constrained nodes to buffer and process data locally, supporting local-first, low-latency processing patterns required in edge and IoT scenarios (AP-I-4) [53].

Kafka’s event-log semantics also underpin key quality attributes such as robustness, auditability, and reproducibility. By emitting every state change or model-related action as an event, the system maintains a detailed history of data transformations, feature computations, and model decisions in dedicated topics, directly supporting full traceability and comprehensive audit trails (AP-DM-7, AP-QA-0, AP-QA-6) [29] [26]. The same event history can be replayed to reproduce model behavior or investigate incidents, thereby satisfying reproducibility requirements for re-running pipelines on identical inputs [37] [27].

Data Orchestrator. Dagster offers an asset-centric orchestration which natively incorporates data-quality and observability features. It provides asset checks that automatically catch and reject malformed or noisy data before downstream tasks [25]. Dagster pipelines can ingest data from many sources (databases, files, streams, APIs) through its resource I/O manager system and connectors, and supports both scheduled runs and event-based sensors (AP-DM-5 and AP-DM-6) [51]. Dagster’s parallel, dynamic execution model scales to large data volumes and heterogeneous data types, addressing three of the traditional “5Vs” of big data (volume, velocity, and variety) . Crucially, it also automatically records data lineage and provenance at every step of the pipeline (AP-DM-7, AP-QA-0) [56].

Dagster serves as a unified orchestration layer (AP-I-0) that coordinates data and machine learning pipelines end-to-end. It can automatically trigger data preparation and model training pipelines when new data arrive or according to a predefined schedule, and its declarative automation makes retraining pipelines repeatable and reliable [24]. Dagster Plus supports Git branch-based deployments, so that each branch can instantiate an isolated staging environment for testing changes (AP-I-3) [21]. Pipelines and their configurations are defined as code, enabling integration with continuous integration and continuous delivery systems, so that changes to data logic or jobs are automatically built, tested, and deployed (AP-I-9) [21].

Dagster ensures reproducibility and traceability (AP-QA-0, QA-6). Its UI and logs capture each asset run together with metadata (such as data versions, code commits, and evaluation metrics), so that reruns with the same inputs deterministically yield the same results. Together, these materialization logs and the asset catalog provide a complete audit trail of all data transformations [56, 23].

Persistent Storage. Taken together, TimescaleDB (a PostgreSQL-based time-series database), ClickHouse (a columnar analytics warehouse), and MinIO (S3-compatible object storage) satisfy the identified data and ML architecture principles through complementary capabilities. Dataset versioning and lineage (AP-DM-0) are natively supported by MinIO’s S3-compatible storage: the system can take snapshots of datasets at each update, ensuring reproducibility and end-to-end lineage control (AP-DM-0) [22].

TimescaleDB is designed for real-time analytics on time-series data and delivers sub-second queries over large volumes of incoming measurements, enabling fast lookups on both recent and historical records [30]. ClickHouse complements this functionality by ingesting high-volume data streams efficiently and accelerating analytical queries, sustaining millions of inserted rows per second with low latency [18]. Together, TimescaleDB and ClickHouse support both streaming and batch ingestion, and their architectures are explicitly optimized for low-latency ingest and querying (AP-DM-5, AP-DM-6). These technologies also address the “5Vs” of big data (AP-DM-8), scaling to extreme volume and velocity.

In practice, MLflow or a comparable experiment-tracking system utilize MinIO as its artifact store and TimescaleDB/PostgreSQL as its backend store (AP-ML-7). Under this shared S3-plus-SQL pattern, experiment metadata and artifacts are stored per run, enabling reproducible experimentation [10, 11].

Training Environment. A training environment combining GitLab (CI/CD and version control), MLflow (experiment tracking and model registry), and DVC (data version control and pipelines) can satisfy the listed architectural principles in several ways. DVC provides strong data versioning and reproducibility (AP-DM-0, AP-QA-0) by storing large datasets in remote storage while keeping lightweight pointers and metadata in GitLab [41], DVC also captures data lineage (AP-DM-7) by declaring dependencies and outputs for each pipeline step [26]. MLflow’s Model Registry guarantees versioned models (AP-ML-0) and links each model version to the run and data that produced it [43]. By combining these tools, the stack inherently enforces reproducibility and traceability (AP-QA-0, AP-QA-6).

GitLab acts as the central orchestration and CI/CD layer (AP-I-0, AP-I-1), automating the end-to-end machine learning pipeline [13]. Its pipelines enforce consistent evaluation: for example, if an evaluation job computes metrics on a holdout set, its results can gate model promotion (AP-ML-2). MLflow’s experiment tracking (or GitLab’s experiment features) records dataset versions, parameters, and metrics for each run (AP-ML-7) [48]. Version history in Git, combined with DVC, also satisfies AP-DM-3 (storing historical data) by retaining past data snapshots for audit or backtesting [41].

MLflow’s tracking server logs each run’s parameters, metrics, and artifacts and exposes them through a web interface, where model experiments can be analyzed (AP-QA-6, AP-QA-7). GitLab integrates with MLflow so that each merge request can trigger a pipeline whose results, including MLflow outputs, appear as merge request notes or in the Analyze menu under Model Experiments [13]. DVC complements this with built-in plotting of metrics across versions, allowing teams to visualize

learning curves without additional tools [41]. Together, these dashboards support expert review and enable informed human intervention (AP-QA-7).

Inference Service. KServe integrates with streaming platforms (Apache Kafka) so that inference requests are triggered by events, decoupling data producers from the model service [27]. An InferenceService can be configured to consume messages from Kafka topics and HTTP endpoints and to serve predictions for both real-time streams and on-demand API requests. Its agent-sidecar batcher can aggregate inference requests into batched jobs, improving efficiency and throughput (AP-DM-2, AP-DM-5, AP-DM-6) [36].

KServe’s InferenceService supports controlled model rollouts by tracking revisions of deployed models and enabling canary release strategies (AP-ML-3) [16, 39]. KServe also exposes observability hooks: it can log inference payloads and integrate with monitoring tools (e.g. Prometheus), as well as with libraries for drift and outlier detection (e.g. Alibi) (AP-ML-5) [4]. InferenceService resources are defined in version-controlled YAML and can be deployed through CI/CD pipelines (e.g. GitOps workflows) (AP-I-9).

Each InferenceService references a specific model URI and runtime (AP-QA-0) [27]. Through KServe’s built-in explainer interface, explainability components (e.g. Alibi or AIX360) can be attached to an InferenceService to obtain feature attributions or decision rationales for each prediction (AP-QA-2) [47]. KServe’s payload-logging feature can record the inputs and outputs of every inference, providing a complete audit trail of model behavior (AP-QA-6) [7].

Medical Visualization Dashboard. A Medical Visualization Dashboard is implemented in Grafana. Grafana is explicitly designed for customizable dashboards, rich visualizations, and fine-grained alerting rules, enabling tailored expert views of IMU sensor data and model predictions (AP-ML-5, AP-QA-7) [35]. These custom visuals support informed decision-making by helping domain experts quickly interpret complex signals, guide timely human intervention when alerts are triggered, and actively oversee live IMU data.

Resource Monitoring. The Resource Monitoring component provides continuous visibility into both the runtime behavior of machine-learning systems and the underlying infrastructure resources that support them. Prometheus is used to collect and store custom application, model, and system-level metrics, including CPU, memory, storage, and latency indicators. Grafana builds on these metrics to deliver real-time dashboards and alerting, enabling operators to monitor resource consumption alongside model performance and quickly identify bottlenecks, failures, or degradation in deployed systems [42].

In combination with Loki, the stack enables unified collection and correlation of metrics and logs across the entire architecture. Loki functions as a scalable log aggregation system, capturing application logs, resource utilization events, data processing records, and model inference outputs in distributed environments. Logs generated by upstream components can be transformed into Prometheus metrics and visualized within Grafana, supporting integrated analysis of system behavior, resource usage, and model execution for improved traceability and root-cause analysis [31].

Metadata Store. The Metadata Store is a foundational component, it provides a centralized system of record for technical, operational, and business metadata across data and machine learning assets. In a distributed, event-driven, microservice-based architecture, metadata acts as the connective tissue that enables coordination, governance, observability, and trust without introducing tight coupling between components.

OpenMetadata, as an open and extensible metadata platform, directly addresses these key architectural challenges. By centralizing lineage, version history, and model metadata, it enhances reproducibility and provides end-to-end transparency and traceability across data pipelines and machine learning lifecycles (AP-QA-0) [19]. Built-in data quality tests, anomaly alerts, lineage visualizations, and drift monitoring contribute to robustness and adaptability by enabling early issue detection and timely intervention (AP-QA-1, AP-qa-5) [19].

OpenMetadata further supports explainability through feature-level and model-level lineage, as well as by storing explainability artifacts and governance documentation (AP-QA-2) [19]. Although it does not compute fairness metrics directly, it provides a structured environment in which such metrics can be documented, monitored, and contextualized within broader governance processes (AP-QA-3) [19]. Finally, collaboration features, dashboards, alerts, and workflow mechanisms ensure that human oversight remains integral, enabling review, intervention, and documentation of decisions (AP-QA-6) [19].

Edge Device. The *Edge Device* (e.g. smartphone, tablet) enables patients to pair IMU sensors using BLE protocol and stream raw data in real time to the *Message Broker*. It performs on-device preprocessing, such as orientation checks, and caches data when internet connectivity is unavailable.

3.3 Proof of Concept

This section describes how the proposed architecture is realized in a proof-of-concept implementation. The proof-of-concept demonstrates the end-to-end flow of IMU data, from real-time acquisition and validation through feature extraction, model inference, and visualization, showing how the architectural components interact in practice to support continuous assessment of functional knee instability.

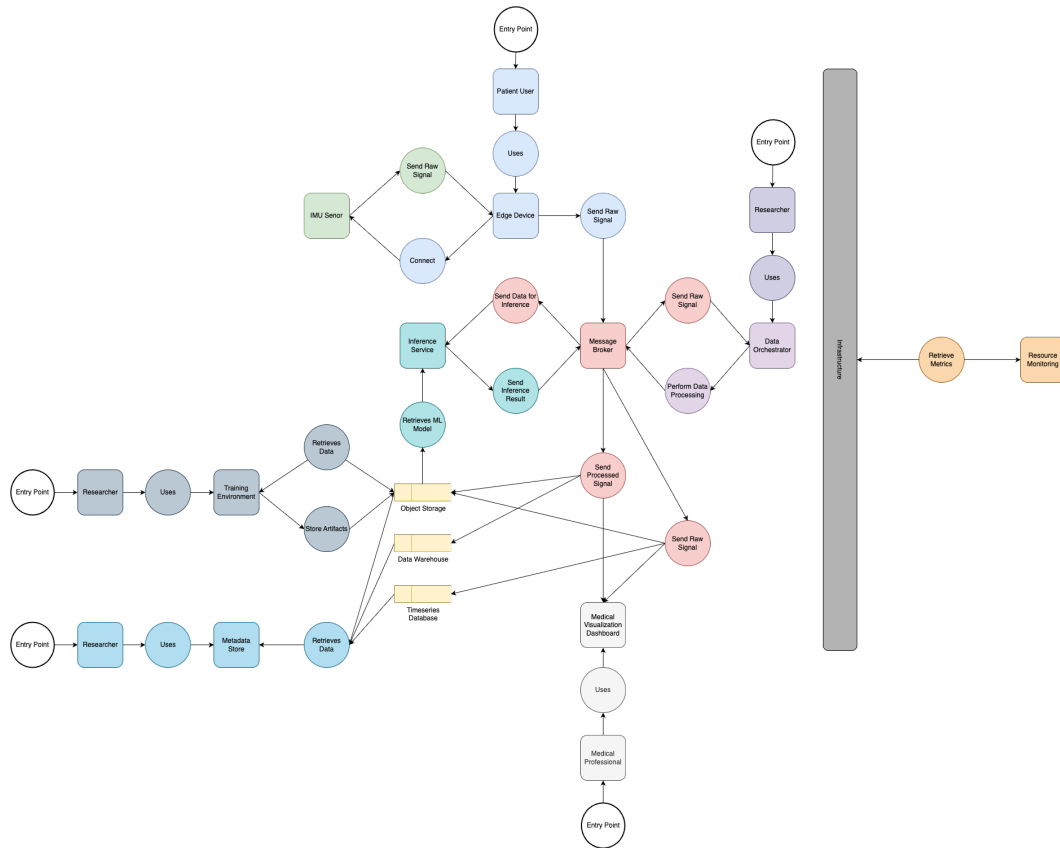


Figure 2: Data Flow Diagram Level 2 shows the end-to-end data flow of the Proof-of-Concept. IMU sensors stream data via an edge device to a message broker, where raw signals are persisted, transformed into features, republished to downstream services, stored for analytics and machine learning experimentation, and visualized for clinical use, with monitoring and metadata captured across the pipeline.

Figure 2 illustrates the data flow of the proposed system utilizing a Data Flow Diagram level 2 [55]. IMU sensors stream raw signals to the user’s *Edge Device*, which forwards the data to the *Message Broker* to ensure reliable delivery. The *Timeseries* database persists the raw incoming IMU data, while the *Data Orchestrator* handles downstream processing, such as signal filtering and feature extraction. The resulting outputs are then republished via the *Message Broker* to subscribed components and stored in the *Datawarehouse* for analytical purposes. In addition, the *Object Store* retains both raw and processed data to support auditing and machine learning experimentation. The *Medical Visualization Dashboard* presents both raw and processed data in an interpretable manner for medical professionals.

In parallel, the *Resource Monitoring* component tracks the resource utilization of architectural components, while the *Metadata Store* maintains comprehensive metadata across the system, enabling traceability and governance.

3.3.1 Functionality

The following figures illustrate three operational flows: 1) data collection and metadata representation, 2) machine learning model development and deployment, 3) and clinical data visualization. Together, these workflows demonstrate how raw sensor data are transformed into clinically meaningful insights in the Proof-of-Concept implementation.

Figure 3 focuses on the translation of processed data into clinician-facing visualizations. After storage and aggregation, selected features and derived metrics are retrieved and rendered within a medical visualization dashboard. This dashboard is intended to present dynamic knee behavior in an interpretable manner, supporting comparison across tasks, or sessions. This is essential for aligning technical outputs with clinical reasoning processes and for ensuring that visualized information supports, professional judgment in assessment of functional knee instability.

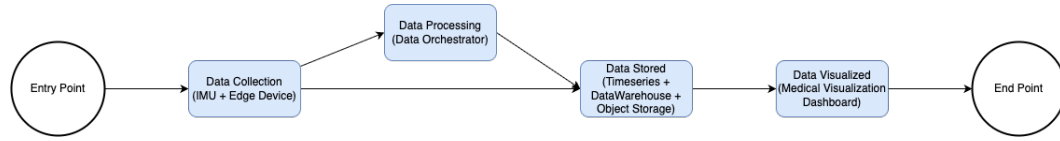


Figure 3: DataOps pipeline illustrating the retrieval of processed data for visualization within a medical dashboard to support clinical interpretation.

Figure 4 illustrates the pipeline responsible for ingesting raw IMU signals collected during functional and sport-specific tasks. After ingestion, preprocessing, and data persistence, the resulting metadata are consolidated in a central dashboard. This dashboard ensures auditability, supports reproducibility, and enables downstream analytical and clinical workflows to reason about data quality and provenance.

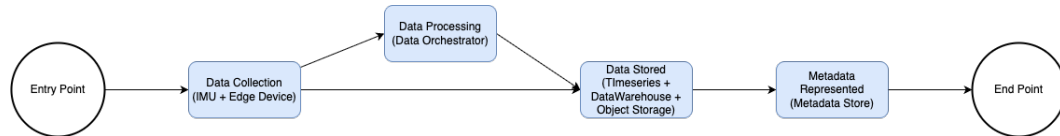


Figure 4: End-to-end data flow from IMU-based data collection through centralized processing and storage, culminating in structured metadata representation for governance and traceability.

Figure 5 presents the pipeline that operationalizes machine learning. Dataset(s) are retrieved from object storage and supplied to the *Training Environment*, where experiments can be tracked, compared, and reproduced. Trained models, along with associated artifacts such as feature definitions and

evaluation metrics, are versioned and stored for governance and rollback. Once validated, selected models are deployed to the *Inference Service* capable of processing real-world and real-time incoming IMU data.

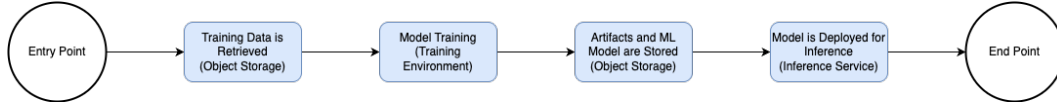


Figure 5: MLOps workflow depicting the retrieval of curated datasets, model training, artifact storage, and deployment of trained models for inference.

4 Discussion

This study proposes a DataOps & MLOps driven architecture capable of streaming, processing, and governing high-frequency IMU data to support clinical assessment of functional knee instability. The proposed architecture responds directly to gaps identified in the literature: 1) IMU-based methods are rarely deployed beyond laboratory environments, and 2) prior systems generally treat sensing, modelling, and data management as isolated components rather than as parts of a production-grade data and ML platform [5, 38, 6]. By Structuring the proposed architecture on DataOps & MLOps architectural principles across four domains: 1) Data Management, 2) the ML Lifecycle, 3) Infrastructure, and 4) Quality Attributes, the proposed architecture is conceptually designed to be 1) scalable and high-performing, 2) flexible and extensible, and 3) resilient and robust.

Prior research has shown that IMUs can reduce examiner subjectivity during clinical laxity assessments, yet these approaches remain constrained to controlled environments and depend heavily on specific maneuvers [57, 12]. Similarly, work on wearable-sensor analytics has focused primarily on gait event detection and estimation of ground reaction forces (GRFs) or joint moments, but with limited validation on pathological gait and considerable heterogeneity in evaluation metrics [50, 6]. These limitations hinder comparability and impede translation to clinical workflows [50]. Reviews further highlight the difficulty of inferring kinetic quantities from kinematics alone and the predominance of studies involving healthy participants, underscoring the need for approaches that can operate in real-world contexts [6].

The effort of this study also align with emerging ACL-specific research that calls for standardized analytics pipelines [38]. While low-cost IMU systems have demonstrated promise in estimating spatiotemporal gait parameters, these implementations generally rely on offline processing and do not support scalable, governed data management [32]. The present architecture advances beyond such prototypes by enabling real-time data ingestion and parallel stream processing. In doing so, it provides a foundation for generating interpretable, traceable metrics that may contribute to a more objective and clinically meaningful definition of functional knee instability.

Against this backdrop, the architecture proposed in this study contributes a unified, production-oriented architecture that integrates ingestion, quality control, feature extraction, model training, and model deployment within a governed data environment. By embedding DataOps and MLOps principles into an architecture and proof-of-concept, this work demonstrates how wearable IMU based knee assessment can be advanced beyond isolated experiments toward deployable, end to end clinical systems. Rather than focusing on individual modeling components, the proposed approach operationalizes the full lifecycle from data acquisition to monitoring, enabling real world clinical use that extends beyond controlled laboratory environments and beyond a single algorithm or dataset.

4.1 Architecture Capabilities

The proposed platform offers scalable, high-performance processing of IMU data through a microservice and messaging architecture, enabling independent service scaling and parallel stream

processing. Its loosely coupled, independently deployable services support rapid extension with new functionality or data sources while maintaining stability and consistent communication via shared versioned schemas. Asynchronous messaging, redundancy, and Dagster’s orchestration enhance resilience by preventing data loss and cascading failures. Automated data and model pipelines ensure reliable, reproducible deployments and continuous improvement. Centralized data governance, edge preprocessing, and combined real-time and batch processing together deliver high-quality, traceable data for both immediate clinical monitoring and offline analytical workloads.

4.2 Clinical Impact

The clinical impact stems from its reliable, traceable, and clinician centered design. High availability, message buffering, and Kubernetes self-healing ensure continuous operation and prevent data loss, even under partial failures. Comprehensive traceability and auditability are achieved by storing raw and processed data with timestamps and version tags, and by capturing full data and model lineage. Strict versioning of code, data, and models further supports reproducibility, allowing analyses and decision metrics to be rerun exactly for validation or research. Expert oversight remains central, interactive Grafana dashboards expose raw signals and derived metrics, enabling clinicians to review trends. Finally, low-latency data flows provide near-real-time indicators of functional knee instability, enabling earlier detection of concerning changes and more timely interventions during rehabilitation.

4.3 Organizational Impact

The proposed architecture delivers clear organizational benefits by improving maintainability, compliance, and collaboration. A microservice architecture with clear separation of concerns, combined with containerization and Infrastructure-as-Code, allows independent upgrades of components and reproducible deployments across development, testing, and production environments, reducing configuration drift and downtime. Comprehensive logging, versioning, and audit trails across Kafka, databases, and MLflow support regulatory compliance by documenting when patient data are collected and how models are applied. DataOps and MLOps practices enable continuous improvement through performance monitoring, semi-automatic retraining workflows, and safe A/B testing of new models, allowing the system to incorporate emerging clinical knowledge rapidly. Shared schemas, repositories, and CI/CD pipelines provide a common framework for data engineers, ML researchers, and clinicians, streamlining handovers and fostering sustained cross-team collaboration on the platform’s evolution.

4.4 Limitations

To date, the proposed architecture has been evaluated through stakeholder review, systematic alignment with established DataOps and MLOps design principles, and a technical proof-of-concept implementation. It has not yet been validated in a clinical trial or deployed in routine clinical practice, and the current proof of concept implements only a subset of the identified architectural principles. As a result, several components remain conceptual, and the practical effectiveness of the architecture has not yet been assessed against explicit, measurable criteria. These limitations highlight the exploratory nature of the present work and the need for further empirical validation.

In addition to these maturity constraints, the architecture raises important security and ethical considerations, as it processes sensitive health-related data derived from wearable IMU sensors. Raw signals, derived features, and inferred instability metrics constitute personal health data and may be indirectly identifying, which introduces risks related to unauthorized access, data leakage, or misuse. Moreover, the system is explicitly intended as a clinical decision-support tool rather than an autonomous diagnostic system. Given the absence of a universally accepted ground truth for functional knee instability, model outputs must be interpreted with caution to avoid over-reliance on algorithmic assessments. Ethical compliance therefore requires informed consent, transparency regarding data usage, alignment with data protection regulations, and sustained human oversight. Continuous monitoring may also influence patient well-being by increasing anxiety or perceived surveillance, underscoring the importance of clear clinical protocols and responsible deployment. Ad-

addressing these considerations is essential to ensure that the proposed architecture supports trustworthy, ethical, and clinically meaningful use as it progresses toward real-world application.

5 Conclusion

In this study, an architectural design is proposed for a platform that uses IMUs to capture real-time, task-level measures of knee related function during daily and sport-specific activities, while also supporting research-grade data collection. By embedding DataOps and MLOps architectural design principles that prioritize robust data ingestion and validation, event-aligned feature extraction, and clinician-in-the-loop interpretation, the architecture provides a scalable and high-performing, flexible and extensible, and resilient and robust basis for generating standardized, interpretable metrics to inform diagnosis and recovery monitoring. In doing so, the work contributes a production-oriented architecture that can support objective, real-world evidence for clinical decisions about functional knee instability.

6 Acknowledgments

We would like to express my sincere gratitude to Jorge Uquillas Paredes, and Tim Gerbrands for their guidance throughout this research. Their collective feedback has undeniably enhanced the quality of this work.

References

- [1] *A Reference Architecture for Governance of Cloud Native Applications*. URL: <https://arxiv.org/html/2302.11617v2> (visited on 11/19/2025).
- [2] Kirstin Ahmed et al. “Validation of IMU against optical reference and development of open-source pipeline: proof of concept case report in a participant with transfemoral amputation fitted with a Percutaneous Osseointegrated Implant”. In: *Journal of NeuroEngineering and Rehabilitation* 21.1 (July 2024), p. 128. ISSN: 1743-0003. DOI: 10.1186/s12984-024-01426-6. URL: <https://doi.org/10.1186/s12984-024-01426-6> (visited on 09/24/2025).
- [3] *AI/ML in Kubernetes Best Practices: The Essentials* | Wiz. en-us. Mar. 2025. URL: <https://www.wiz.io/academy/ai-ml-kubernetes-best-practices> (visited on 11/19/2025).
- [4] *Alibi Outlier/Drift Detection* | KServe. en. URL: <https://kserve.github.io/website/docs/model-serving/predictive-inference/detect/alibi/alibi-detect> (visited on 11/24/2025).
- [5] Natasha E. H. Allott et al. “The methods of quantifying knee laxity in the ACL injured population: A review”. In: *The Knee* 55 (Aug. 2025), pp. 85–103. ISSN: 0968-0160. DOI: 10.1016/j.knee.2025.04.009. URL: <https://www.sciencedirect.com/science/article/pii/S0968016025000808> (visited on 10/06/2025).
- [6] Andrea Ancillao et al. “Indirect Measurement of Ground Reaction Forces and Moments by Means of Wearable Inertial Sensors: A Systematic Review”. en. In: *Sensors* 18.8 (Aug. 2018). Publisher: Multidisciplinary Digital Publishing Institute, p. 2564. ISSN: 1424-8220. DOI: 10.3390/s18082564. URL: <https://www.mdpi.com/1424-8220/18/8/2564> (visited on 09/17/2025).
- [7] *Announcing KServe v0.11* | KServe. en. Oct. 2023. URL: <https://kserve.github.io/website/blog/kserve-0.11-release> (visited on 11/24/2025).
- [8] Joseph Anthony. *A Comprehensive Guide to Scaling Microservices*. en. Dec. 2024. URL: <https://crossasyst.com/blog/scaling-microservices/> (visited on 11/19/2025).
- [9] *Apache Kafka*. en. URL: <https://kafka.apache.org/> (visited on 10/01/2025).

- [10] *Artifact Stores* | *MLflow*. en. URL: <https://mlflow.org/docs/latest/self-hosting/architecture/artifact-store/> (visited on 11/24/2025).
- [11] *Backend Stores* | *MLflow*. en. URL: <https://mlflow.org/docs/latest/self-hosting/architecture/backend-store/> (visited on 11/24/2025).
- [12] Paolo Bellitti et al. “Smart Brace for Static and Dynamic Knee Laxity Measurement”. en. In: *Sensors* 22.15 (Jan. 2022). Publisher: Multidisciplinary Digital Publishing Institute, p. 5815. ISSN: 1424-8220. DOI: 10.3390/s22155815. URL: <https://www.mdpi.com/1424-8220/22/15/5815> (visited on 10/08/2025).
- [13] *Build an ML app pipeline with GitLab Model Registry using MLflow*. en-US. URL: <https://about.gitlab.com/blog/build-an-ml-app-pipeline-with-gitlab-model-registry-using-mlflow/> (visited on 11/24/2025).
- [14] *Building Microservices, 1st Edition*. Building Microservices, 1st Edition. URL: https://samnewman.io/books/building_microservices/ (visited on 01/14/2026).
- [15] Agile Business. *MoSCoW Prioritisation - DSDM Project Framework Handbook* | *Agile Business Consortium*. URL: <https://www.agilebusiness.org/dsdm-project-framework/moscow-prioritisation.html> (visited on 12/05/2025).
- [16] *Canary Rollout Strategy* | *KServe*. en. URL: <https://kserve.github.io/website/docs/model-serving/predictive-inference/rollout-strategies/canary> (visited on 11/24/2025).
- [17] Meike Chabowski. *Microservices at Edge with K3s and Fleet*. en-US. URL: <https://www.suse.com/c/microservices-at-edge-with-k3s-and-fleet/> (visited on 11/19/2025).
- [18] ClickHouse. *Building StockHouse: Real-time market analytics with ClickHouse*. en. URL: <https://clickhouse.com/blog/building-stockhouse> (visited on 11/28/2025).
- [19] OpenMetadata Community. *OpenMetadata Standards - Open Standard for Unified Metadata Management*. OpenMetadata Standards. URL: <https://openmetadatastandards.org/> (visited on 01/02/2026).
- [20] Cathelijne Cramer. *Anterior Cruciate Ligament Treatment* | *Physi.nl*. en-GB. Feb. 2023. URL: <https://www.fysi.nl/en/blog/anterior-cruciate-ligament-treatment/> (visited on 10/29/2025).
- [21] *Dagster Supports Every Dev Lifecycle Step*. en. URL: <https://dagster.io/blog/dagster-five-stages-development-lifecycle> (visited on 11/19/2025).
- [22] *Data Authenticity and Integrity in MinIO*. en. Nov. 2021. URL: <https://blog.min.io/data-authenticity-integrity/> (visited on 11/21/2025).
- [23] *Data Orchestration Tools: 10 Key Features & 10 Platforms to Know* | *Dagster Guides*. en. URL: <https://dagster.io/guides/data-orchestration-tools-10-key-features-10-platforms-to-know> (visited on 11/19/2025).
- [24] *Data Pipeline Architecture: 5 Design Patterns with Examples* | *Dagster Guides*. en. URL: <https://dagster.io/guides/data-pipeline-architecture-5-design-patterns-with-examples> (visited on 11/19/2025).
- [25] *Data Quality with Dagster + GX*. URL: <https://dagster.io/blog/ensuring-data-quality-with-dagster-and-great-expectations> (visited on 11/19/2025).
- [26] *dvc.yaml Files*. en. URL: <https://doc.dvc.org/user-guide/project-structure/dvcyaml-files> (visited on 11/28/2025).
- [27] Jakob Edding. *Scalable Machine Learning with Kafka Streams and KServe*. en. Sept. 2023. URL: <https://medium.com/bakdata/scalable-machine-learning-with-kafka-streams-and-kserve-85308858d867> (visited on 11/24/2025).
- [28] Sander van Eijck, Tim Gerbrands, and Petra Heck. *Master of Applied IT Group project case - Measuring dynamic knee laxity in anterior cruciate ligament patients*. (Visited on 09/10/2025).

- [29] *Event Sourcing: An Introduction*. en. URL: <https://www.confluent.io/learn/event-sourcing/> (visited on 11/19/2025).
- [30] Michael J Freedman and James Blackwood-Sewell. "TigerData Architecture for Real-Time Analytics". en. In: ().
- [31] *Grafana Loki OSS | Log aggregation system*. en. URL: <https://grafana.com/oss/loki/> (visited on 11/24/2025).
- [32] Trupti Gujarathi and Kalyani Bhole. "GAIT ANALYSIS USING IMU SENSOR". In: *2019 10th International Conference on Computing, Communication and Networking Technologies (ICCCNT)*. July 2019, pp. 1–5. DOI: 10.1109/ICCCNT45670.2019.8944545. URL: <https://ieeexplore.ieee.org/document/8944545> (visited on 09/17/2025).
- [33] Petra Heck et al. "An Approach for Integrated Development of an MLOps Architecture". en. In: ().
- [34] *Home*. C4 model. URL: <https://c4model.com/> (visited on 01/14/2026).
- [35] *How BasisAI uses Grafana and Prometheus to monitor model drift in machine learning workloads*. en. URL: <https://grafana.com/blog/2021/08/02/how-basisai-uses-grafana-and-prometheus-to-monitor-model-drift-in-machine-learning-workloads/> (visited on 11/24/2025).
- [36] *Inference Batchier | KServe*. en. URL: <https://kserve.github.io/website/docs/model-serving/predictive-inference/batcher> (visited on 11/24/2025).
- [37] *Kafka - replaying events*. en. Oct. 2023. URL: <https://dev.to/easytony/kafka-replaying-events-12ik> (visited on 11/19/2025).
- [38] Sanchana Krishnakumar et al. "Estimation of Kinetics Using IMUs to Monitor and Aid in Clinical Decision-Making during ACL Rehabilitation: A Systematic Review". In: *Sensors (Basel, Switzerland)* 24.7 (Mar. 2024), p. 2163. ISSN: 1424-8220. DOI: 10.3390/s24072163. URL: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC11014074/> (visited on 09/17/2025).
- [39] *KServe*. en. URL: <https://kserve.github.io/website/> (visited on 10/01/2025).
- [40] *Machine Learning Systems: A Survey from a Data-Oriented Perspective*.
- [41] Ramkrushna Maheshwar. *Building Reproducible ML Pipelines with DVC and GitLab*. en. June 2025. URL: <https://medium.com/@maheshwar.ramkrushna/building-reproducible-ml-pipelines-with-dvc-and-gitlab-0ab768eadb89> (visited on 11/24/2025).
- [42] Alan McGinlay. *Infrastructure monitoring: An introduction*. Medium. Oct. 8, 2021. URL: <https://mrintegrity.medium.com/monitoring-from-scratch-ea2b83a8f8a5> (visited on 01/02/2026).
- [43] *MLflow Model Registry | MLflow*. en. URL: <https://mlflow.org/docs/latest/ml/model-registry/> (visited on 11/24/2025).
- [44] *MLOps: Continuous delivery and automation pipelines in machine learning | Cloud Architecture Center*. en. URL: <https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning> (visited on 10/06/2025).
- [45] Tim Mucci, Mark Scapicchio, and Cole Stryker. *What is Dataops? | IBM*. en. Apr. 2024. URL: <https://www.ibm.com/think/topics/dataops> (visited on 11/26/2025).
- [46] Tim Mucci and Cole Stryker. *What is MLOps? | IBM*. en. Apr. 2024. URL: <https://www.ibm.com/think/topics/mlops> (visited on 11/26/2025).
- [47] *Overview | KServe*. en. URL: <https://kserve.github.io/website/docs/model-serving/predictive-inference/explainers/overview> (visited on 11/24/2025).

- [48] Adithya Prasad Pandelu. *Day 66: Data Versioning and Management — DVC, MLflow*. en. Jan. 2025. URL: <https://medium.com/@bhatadithya54764118/day-66-data-versioning-and-management-dvc-mlflow-2d53fe3232ee> (visited on 11/24/2025).
- [49] *Physical activity*. en. URL: <https://www.who.int/news-room/fact-sheets/detail/physical-activity> (visited on 10/29/2025).
- [50] Hari Prasanth et al. “Wearable Sensor-Based Real-Time Gait Detection: A Systematic Review”. en. In: *Sensors* 21.8 (Jan. 2021). Publisher: Multidisciplinary Digital Publishing Institute, p. 2727. ISSN: 1424-8220. DOI: 10.3390/s21082727. URL: <https://www.mdpi.com/1424-8220/21/8/2727> (visited on 09/24/2025).
- [51] *schedules and sensors* | *Dagster Docs*. en. URL: <https://docs.dagster.io/api/dagster/schedules-sensors> (visited on 11/19/2025).
- [52] *Schema Registry for Confluent Platform* | *Confluent Documentation*. URL: https://docs.confluent.io/platform/current/schema-registry/index.html?utm_source=chatgpt.com (visited on 11/19/2025).
- [53] *Unlocking the Edge: Data Streaming Goes Where You Go with Confluent*. en. URL: <https://www.confluent.io/blog/data-streaming-at-the-edge/> (visited on 11/19/2025).
- [54] Stephen John Warnett and Uwe Zdun. “Architectural Design Decisions for the Machine Learning Workflow”. In: *Computer* 55.3 (Mar. 2022), pp. 40–51. ISSN: 1558-0814. DOI: 10.1109/MC.2021.3134800. URL: <https://ieeexplore.ieee.org/document/9734278> (visited on 10/06/2025).
- [55] *What Is a Data Flow Diagram (DFD)?* | *IBM*. en. Nov. 2024. URL: <https://www.ibm.com/think/topics/data-flow-diagram> (visited on 10/27/2025).
- [56] *What Is Lineage* | *Dagster*. en. URL: <https://dagster.io/glossary/data-lineage> (visited on 11/19/2025).
- [57] STEFANO ZAFFAGNINI et al. “Inertial sensors to quantify the pivot shift test in the treatment of anterior cruciate ligament injury”. In: *Joints* 2.3 (Aug. 2014), pp. 124–129. ISSN: 2282-4324. DOI: 10.11138/jts/2014.2.3.124. URL: <https://pmc.ncbi.nlm.nih.gov/articles/PMC4295682/> (visited on 09/24/2025).

A Reviewed Artifacts

In Table 1 are listed all the reviewed artifacts including academic literature, industry reports, company documents, and technical blogs that were reviewed during the study.

NO.	Bibliography
[1]	“Why Microservices Need Event-Driven Architectures for Agility and Scale,” Confluent. Accessed: Nov. 05, 2025. [Online]. Available: https://www.confluent.io/blog/do-microservices-need-event-driven-architectures/
[2]	L. Uhlenberg and O. Amft, “Where to mount the IMU? Validation of joint angle kinematics and sensor selection for activities of daily living,” <i>Frontiers in Computer Science</i> , vol. 6, 2024, doi: 10.3389/fcomp.2024.1347424.
[3]	“What is Unified Modeling Language (UML)?” Accessed: Oct. 27, 2025. [Online]. Available: https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-uml/
[4]	T. Mucci and C. Stryker, “What is MLOps? IBM,” What is machine learning operations (MLOps)? Accessed: Nov. 26, 2025. [Online]. Available: https://www.ibm.com/think/topics/mlops
[5]	“What is MLOps? Databricks,” Databricks. Accessed: Nov. 26, 2025. [Online]. Available: https://www.databricks.com/glossary/mlops

-
- [6] "What Is Lineage | Dagster." Accessed: Nov. 19, 2025. [Online]. Available: <https://dagster.io/glossary/data-lineage>
-
- [7] "What Is Lambda Architecture?," Databricks. Accessed: Nov. 05, 2025. [Online]. Available: <https://www.databricks.com/glossary/lambda-architecture>
-
- [8] "What Is Human In The Loop," Google Cloud. Accessed: Nov. 05, 2025. [Online]. Available: <https://cloud.google.com/discover/human-in-the-loop>
-
- [9] T. Mucci, M. Scapicchio, and C. Stryker, "What is Dataops? | IBM," What is DataOps? Accessed: Nov. 26, 2025. [Online]. Available: <https://www.ibm.com/think/topics/dataops>
-
- [10] "What Is Data Lineage? | IBM." Accessed: Sept. 25, 2025. [Online]. Available: <https://www.ibm.com/think/topics/data-lineage>
-
- [11] "What is Big Data? | IBM." Accessed: Nov. 05, 2025. [Online]. Available: <https://www.ibm.com/think/topics/big-data>
-
- [12] "What Is a Data Flow Diagram (DFD)? | IBM." Accessed: Oct. 27, 2025. [Online]. Available: <https://www.ibm.com/think/topics/data-flow-diagram>
-
- [13] "What does the EU GDPR mean for your medical device product?" Accessed: Oct. 08, 2025. [Online]. Available: <https://www.extrahorizon.com/gdpr-software-medtech-medical-device-eu-regulation-mdr-data-privacy>
-
- [14] H. Prasanth et al., "Wearable Sensor-Based Real-Time Gait Detection: A Systematic Review," *Sensors*, vol. 21, no. 8, p. 2727, Jan. 2021, doi: 10.3390/s21082727.
-
- [15] C. Dinh-Le, R. Chuang, S. Chokshi, and D. Mann, "Wearable Health Technology and Electronic Health Record Integration: Scoping Review and Future Directions," *JMIR Mhealth Uhealth*, vol. 7, no. 9, p. e12861, Sept. 2019, doi: 10.2196/12861.
-
- [16] Google Cloud, "Vertex AI Model Monitoring." [Online]. Available: <https://cloud.google.com/vertex-ai/docs/model-monitoring>
-
- [17] K. Ahmed, S. Taheri, I. Weygers, and M. Ortiz-Catalan, "Validation of IMU against optical reference and development of open-source pipeline: proof of concept case report in a participant with transfemoral amputation fitted with a Percutaneous Osseointegrated Implant," *Journal of NeuroEngineering and Rehabilitation*, vol. 21, no. 1, p. 128, July 2024, doi: 10.1186/s12984-024-01426-6.
-
- [18] "Validate data schema with GX | Great Expectations." Accessed: Sept. 24, 2025. [Online]. Available: https://docs.greatexpectations.io/docs/reference/learn/data_quality_use_cases/schema/
-
- [19] D.-S. Komaris, G. Tarfali, B. O'Flynn, and S. Tedesco, "Unsupervised IMU-based evaluation of at-home exercise programmes: a feasibility study," *BMC Sports Sci Med Rehabil*, vol. 14, p. 28, Feb. 2022, doi: 10.1186/s13102-022-00417-1.
-
- [20] "Unlocking the Edge: Data Streaming Goes Where You Go with Confluent," Confluent. Accessed: Nov. 19, 2025. [Online]. Available: <https://www.confluent.io/blog/data-streaming-at-the-edge/>
-
- [21] freeCodeCamp.org, UML Diagrams Full Course (Unified Modeling Language), (Apr. 21, 2021). Accessed: Oct. 27, 2025. [Online Video]. Available: <https://www.youtube.com/watch?v=WnMQ8HlmeXc>
-
- [22] DSwithBappy, Ultimate MLOps Full Course in One Video, (June 12, 2025). Accessed: Oct. 01, 2025. [Online Video]. Available: <https://www.youtube.com/watch?v=w71RHxAWxaM>
-
- [23] B. Li et al., "Trustworthy AI: From Principles to Practices," *ACM Comput. Surv.*, vol. 55, no. 9, pp. 1-46, Sept. 2023, doi: 10.1145/3555803.
-

-
- [24] T. lakeFS Team, "Top 17 Data Orchestration Tools for 2025: Ultimate Review," lakeFS. Accessed: Oct. 01, 2025. [Online]. Available: <https://lakefs.io/blog/data-orchestration-tools-2023/>
-
- [25] M. J. Freedman and J. Blackwood-Sewell, "TigerData Architecture for Real-Time Analytics".
-
- [26] J. I. Sanchez, M. Plaza, and N. Echeverria, "Three-Dimensional Electrogoniometry Device and Methods for Measuring and Characterizing Knee Mobility and Multi Directional Instability During Gait," *Biomechanics*, vol. 5, no. 3, p. 68, Sept. 2025, doi: 10.3390/biomechanics5030068.
-
- [27] N. E. H. Allott, F. T. Oladipo, K. L. Cox, C. M. Finnerty, M. S. Banger, and A. H. McGregor, "The methods of quantifying knee laxity in the ACL injured population: A review," *The Knee*, vol. 55, pp. 85-103, Aug. 2025, doi: 10.1016/j.knee.2025.04.009.
-
- [28] freeCodeCamp.org, System Design Concepts Course and Interview Prep, (July 25, 2024). Accessed: Oct. 28, 2025. [Online Video]. Available: https://www.youtube.com/watch?v=F2FmTdLtb_4
-
- [29] P. Bellitti, M. Borghetti, N. F. Lopomo, E. Sardini, and M. Serpelloni, "Smart Brace for Static and Dynamic Knee Laxity Measurement," *Sensors*, vol. 22, no. 15, p. 5815, Jan. 2022, doi: 10.3390/s22155815.
-
- [30] T. Lehmann, L. Paschen, and J. Baumeister, "Single-Leg Assessment of Postural Stability After Anterior Cruciate Ligament Injury: a Systematic Review and Meta-Analysis," *Sports Med - Open*, vol. 3, no. 1, p. 32, Aug. 2017, doi: 10.1186/s40798-017-0100-5.
-
- [31] Seldon, "Seldon Deploy: Model Monitoring and Alerts." [Online]. Available: <https://docs.seldon.io/projects/seldon-core/en/latest/>
-
- [32] "Schema Registry for Confluent Platform | Confluent Documentation." Accessed: Nov. 19, 2025. [Online]. Available: https://docs.confluent.io/platform/current/schema-registry/index.html?utm_source=chatgpt.com
-
- [33] "schedules and sensors | Dagster Docs." Accessed: Nov. 19, 2025. [Online]. Available: <https://docs.dagster.io/api/dagster/schedules-sensors>
-
- [34] "Scale Machine Learning & AI Computing | Ray by Anyscale," Scale Machine Learning & AI Computing | Ray by Anyscale. Accessed: Oct. 01, 2025. [Online]. Available: <https://ray.io>
-
- [35] J. Edding, "Scalable Machine Learning with Kafka Streams and KServe," bakdata. Accessed: Nov. 24, 2025. [Online]. Available: <https://medium.com/bakdata/scalable-machine-learning-with-kafka-streams-and-kserve-85308858d867>
-
- [36] "Sam Newman Building Microservices." Accessed: Nov. 11, 2025. [Online]. Available: https://samnewman.io/books/building_microservices/
-
- [37] "SageMotion - How Do Euler Angles Work?," SageMotion. Accessed: Oct. 03, 2025. [Online]. Available: <https://www.sagemotion.com/blog/how-do-euler-angles-work>
-
- [38] "RabbitMQ: One broker to queue them all | RabbitMQ." Accessed: Oct. 01, 2025. [Online]. Available: <https://www.rabbitmq.com/>
-
- [39] "Pro's and con's of supervised vs unsupervised algorithms for scalable anomaly detection." Accessed: Oct. 07, 2025. [Online]. Available: <https://www.eyer.ai/blog/pros-and-cons-of-supervised-vs-unsupervised-algorithms-for-scalable-anomaly-detection/>
-
- [40] "Physical activity." Accessed: Oct. 29, 2025. [Online]. Available: <https://www.who.int/news-room/fact-sheets/detail/physical-activity>
-

-
- [41] S. M, “Peer-to-Peer (P2P) Architecture: Decentralized by Design,” Medium. Accessed: Nov. 05, 2025. [Online]. Available: <https://simsonmoses.medium.com/peer-to-peer-p2p-architecture-decentralized-by-design-3f73e37048fd>
 - [42] “Overview | KServe.” Accessed: Nov. 24, 2025. [Online]. Available: <https://kserve.github.io/website/docs/model-serving/predictive-inference/explainers/overview>
 - [43] O. Community, “Overview,” OpenMetadata Standards. Accessed: Jan. 02, 2026. [Online]. Available: <https://openmetadatastandards.org/data-assets/ml/overview/>
 - [44] “OpenTofu.” Accessed: Nov. 03, 2025...?
 - [45] freeCodeCamp.org, OpenStack Tutorial – Operate Your Own Private Cloud (Full Course), (July 11, 2022). Accessed: Nov. 03, 2025. [Online Video]. Available: https://www.youtube.com/watch?v=_gWfFEuert8
 - [46] O. Community, “OpenMetadata Standards - Open Standard for Unified Metadata Management,” OpenMetadata Standards. Accessed: Jan. 02, 2026. [Online]. Available: <https://openmetadatastandards.org/>
 - [47] “OpenMetadata Features | Product Overview.” Accessed: Jan. 02, 2026. [Online]. Available: <https://docs.open-metadata.org/latest/features>
 - [48] “Open Source Cloud Computing Infrastructure - OpenStack.” Accessed: Nov. 03, 2025. [Online]. Available: <https://www.openstack.org/>
 - [49] J. C. Schrijvers, J. C. Van Den Noort, M. Van Der Esch, and J. Harlaar, “Neuro-mechanical assessment of knee joint instability during perturbed gait in patients with knee osteoarthritis,” *Journal of Biomechanics*, vol. 118, p. 110325, Mar. 2021, doi: 10.1016/j.jbiomech.2021.110325.
 - [50] G. P. M. Pinheiro et al., “Multi-Sensor Wearable Health Device Framework for Real-Time Monitoring of Elderly Patients Using a Mobile Application and High-Resolution Parameter Estimation,” *Front Hum Neurosci*, vol. 15, p. 750591, 2021, doi: 10.3389/fnhum.2021.750591.
 - [51] A. Business, “MoSCoW Prioritisation - DSDM Project Framework Handbook | Agile Business Consortium.” Accessed: Dec. 05, 2025. [Online]. Available: <https://www.agilebusiness.org/dsdm-project-framework/moscow-prioritisation.html>
 - [52] “Monitoring machine learning models in production with Grafana and ClearML,” Grafana Labs. Accessed: Nov. 24, 2025. [Online]. Available: <https://grafana.com/blog/2023/08/18/monitoring-machine-learning-models-in-production-with-grafana-and-clearml/>
 - [53] “Modern Data Orchestrator Platform | Dagster.” Accessed: Oct. 01, 2025. [Online]. Available: <https://dagster.io>
 - [54] “Model Registry Beta (#9423) · Epics · Epics · GitLab.org · GitLab,” GitLab. Accessed: Nov. 11, 2025. [Online]. Available: <https://gitlab.com/groups/gitlab-org/-/epics/9423>
 - [55] “Model Explainability - Amazon SageMaker AI.” Accessed: Nov. 05, 2025. [Online]. Available: <https://docs.aws.amazon.com/sagemaker/latest/dg/clarify-model-explainability.html>
 - [56] “MLOps: Continuous delivery and automation pipelines in machine learning | Cloud Architecture Center,” Google Cloud Documentation. Accessed: Nov. 05, 2025. [Online]. Available: <https://docs.cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning>
 - [57] “MLOps: Continuous delivery and automation pipelines in machine learning | Cloud Architecture Center,” Google Cloud. Accessed: Oct. 06, 2025. [Online]. Available: <https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning>
-

-
- [58] freeCodeCamp.org, MLOps Pipeline with Python, AWS, Docker – YouTube Viewer Sentiment, (June 13, 2025). Accessed: Oct. 01, 2025. [Online Video]. Available: <https://www.youtube.com/watch?v=gwNPV882tkc>
-
- [59] N. Hystax, “MLOps maturity levels: the most well-known models,” Medium. Accessed: Oct. 28, 2025. [Online]. Available: <https://medium.com/@NickHystax/mlops-maturity-levels-the-most-well-known-models-5b1de94ea285>
-
- [60] D. George, “MLOps in Healthcare: Better Models and Faster Results,” Hakkoda. Accessed: Nov. 26, 2025. [Online]. Available: <https://hakkoda.io/resources/mlops-in-healthcare/>
-
- [61] O. X. Hong, “MLOps 02: Dagster — Seamless Data Pipelines & ML Model Management for Machine Learning Engineers,” Medium. Accessed: Nov. 03, 2025. [Online]. Available: <https://medium.com/@ongxuanhong/mlops-02-dagster-seamless-data-pipelines-ml-model-management-for-machine-learning-engineers-92afb18c3628>
-
- [62] “MLOps | GitLab Docs.” Accessed: Nov. 05, 2025. [Online]. Available: <https://docs.gitlab.com/user/project/ml/>
-
- [63] “MLflow: A Tool for Managing the Machine Learning Lifecycle | MLflow.” Accessed: Nov. 05, 2025. [Online]. Available: <https://mlflow.org/docs/3.1.3/ml/>
-
- [64] “MLflow Model Registry | MLflow.” Accessed: Nov. 24, 2025. [Online]. Available: <https://mlflow.org/docs/latest/ml/model-registry/>
-
- [65] “MLflow Model Registry | MLflow.” Accessed: Nov. 05, 2025. [Online]. Available: <https://mlflow.org/docs/latest/ml/model-registry/>
-
- [66] “MLflow.” Accessed: Oct. 01, 2025. [Online]. Available: <http://mlflow.org/>
-
- [67] M. Chabowski, “Microservices at Edge with K3s and Fleet.” Accessed: Nov. 19, 2025. [Online]. Available: <https://www.suse.com/c/microservices-at-edge-with-k3s-and-fleet/>
-
- [68] RobBagby, “Microservices Architecture Style - Azure Architecture Center.” Accessed: Nov. 05, 2025. [Online]. Available: <https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/microservices>
-
- [69] S. van Eijck, T. Gerbrands, and P. Heck, “Master of Applied IT Group project case - Measuring dynamic knee laxity in anterior cruciate ligament patients.”
-
- [70] “Magical Data Engineering Workflows | Mage AI,” Mage. Accessed: Oct. 01, 2025. [Online]. Available: <https://www.mage.ai>
-
- [71] “Machine Learning Systems: A Survey from a Data-Oriented Perspective.”
-
- [72] T. S. Boudard Léopold, “Machine learning model monitoring: Best practices,” Datadog. Accessed: Nov. 05, 2025. [Online]. Available: <https://www.datadoghq.com/blog/ml-model-monitoring-in-production-best-practices/>
-
- [73] “Kubeflow,” Kubeflow. Accessed: Oct. 01, 2025. [Online]. Available: <https://www.kubeflow.org/>
-
- [74] “KServe.” Accessed: Oct. 01, 2025. [Online]. Available: <https://kserve.github.io/website/>
-
- [75] N. PS, “Kafka replication and fault tolerance,” Medium. Accessed: Nov. 05, 2025. [Online]. Available: <https://medium.com/@psnavya90/kafka-replication-and-fault-tolerance-144367ae38e7>
-
- [76] “Kafka - replaying events,” DEV Community. Accessed: Nov. 19, 2025. [Online]. Available: <https://dev.to/easytony/kafka-replaying-events-12ik>
-
- [77] “JDBC Source and Sink Connector for Confluent Platform | Confluent Documentation.” Accessed: Oct. 20, 2025. [Online]. Available: <https://docs.confluent.io/kafka-connectors/jdbc/current/overview.html>
-

-
- [78] M. W. Creaby, "It's not all about the knee adduction moment: the role of the knee flexion moment in medial knee joint loading," *Osteoarthritis and Cartilage*, vol. 23, no. 7, pp. 1038-1040, July 2015, doi: 10.1016/j.joca.2015.03.032.
-
- [79] Anastasia, "IoT, Edge Computing, and Digitalization in Healthcare," ObjectBox. Accessed: Oct. 08, 2025. [Online]. Available: <https://objectbox.io/iot-edge-computing-and-digitalization-in-healthcare/>
-
- [80] TigerData (creators of TimescaleDB), Introduction to Timescale, (June 13, 2022). Accessed: Oct. 14, 2025. [Online Video]. Available: https://www.youtube.com/watch?v=vm_hcP4CDbA
-
- [81] "Ingest and preprocess data | Dagster Docs." Accessed: Nov. 05, 2025. [Online]. Available: <https://docs.dagster.io/examples/full-pipelines/ml/data-ingestion>
-
- [82] A. McGinlay, "Infrastructure monitoring: An introduction," Medium. Accessed: Jan. 02, 2026. [Online]. Available: <https://mrintegrity.medium.com/monitoring-from-scratch-ea2b83a8f8a5>
-
- [83] "Inference Batchers | KServe." Accessed: Nov. 24, 2025. [Online]. Available: <https://kserve.github.io/website/docs/model-serving/predictive-inference/batcher>
-
- [84] S. ZAFFAGNINI et al., "Inertial sensors to quantify the pivot shift test in the treatment of anterior cruciate ligament injury," *Joints*, vol. 2, no. 3, pp. 124-129, Aug. 2014, doi: 10.11138/jts/2014.2.3.124.
-
- [85] G. Cooper et al., "Inertial sensor-based knee flexion/extension angle estimation," *Journal of Biomechanics*, vol. 42, no. 16, pp. 2678-2685, Dec. 2009, doi: 10.1016/j.jbiomech.2009.08.004.
-
- [86] A. Ancillao, S. Tedesco, J. Barton, and B. O'Flynn, "Indirect Measurement of Ground Reaction Forces and Moments by Means of Wearable Inertial Sensors: A Systematic Review," *Sensors*, vol. 18, no. 8, p. 2564, Aug. 2018, doi: 10.3390/s18082564.
-
- [87] "incubation-engineering · GitLab," GitLab. Accessed: Nov. 26, 2025. [Online]. Available: <https://gitlab.com/gitlab-org/incubation-engineering>
-
- [88] "Human Activity Recognition - Using Deep Learning Model," Geeks-forGeeks. Accessed: Nov. 11, 2025. [Online]. Available: <https://www.geeksforgeeks.org/deep-learning/human-activity-recognition-using-deep-learning-model/>
-
- [89] "Human Activity Recognition." Accessed: Nov. 11, 2025. [Online]. Available: <https://kaggle.com/code/asmahwimli/human-activity-recognition>
-
- [90] P. Bioengineering, "How to Stream Data from Multiple Movella DOTs with a Mac and Python," Medium. Accessed: Oct. 13, 2025. [Online]. Available: <https://medium.com/@protobioengineering/how-to-stream-data-from-multiple-movella-dots-with-a-mac-and-python-afc388f62fa0>
-
- [91] P. Bioengineering, "How to Stream Data from a Movella DOT Wearable Sensor with a Mac and Python," Medium. Accessed: Sept. 17, 2025. [Online]. Available: <https://medium.com/@protobioengineering/how-to-stream-data-from-a-movella-dot-wearable-sensor-with-a-mac-and-python-5822e76fb43e>
-
- [92] "How to keep up with CI/CD best practices," about.gitlab.com. Accessed: Nov. 05, 2025. [Online]. Available: <https://about.gitlab.com/blog/how-to-keep-up-with-ci-cd-best-practices/>
-
- [93] "How TimescaleDB helped us scale analytics and reporting," The Cloudflare Blog. Accessed: Nov. 21, 2025. [Online]. Available: <https://blog.cloudflare.com/timescaledb-art/>
-

-
- [94] “How I Built Our Own OpenStack Cloud on a Virtual Machine (Step-by-Step Guide),” DEV Community. Accessed: Nov. 05, 2025. [Online]. Available: <https://dev.to/p0saurabh/how-i-built-my-own-openstack-cloud-on-a-virtual-machine-step-by-step-guide-1lj3>
-
- [95] “How BasisAI uses Grafana and Prometheus to monitor model drift in machine learning workloads,” Grafana Labs. Accessed: Nov. 24, 2025. [Online]. Available: <https://grafana.com/blog/2021/08/02/how-basisai-uses-grafana-and-prometheus-to-monitor-model-drift-in-machine-learning-workloads/>
-
- [96] “Home | OpenLineage.” Accessed: Oct. 06, 2025. [Online]. Available: <https://openlineage.io/>
-
- [97] “Home | Movella.com.” Accessed: Oct. 08, 2025. [Online]. Available: <https://www.movella.com>
-
- [98] “Grafana Loki OSS | Log aggregation system,” Grafana Labs. Accessed: Nov. 24, 2025. [Online]. Available: <https://grafana.com/oss/loki/>
-
- [99] “GitLab.org / incubation-engineering / MLOps / Data Science CI Example · GitLab,” GitLab. Accessed: Nov. 11, 2025. [Online]. Available: <https://gitlab.com/gitlab-org/incubation-engineering/mlops/data-science-ci-example>
-
- [100] GitLab Unfiltered, GitLab Model Registry Demo: creating and packaging an Machine Learning model, (Feb. 16, 2024). Accessed: Nov. 11, 2025. [Online Video]. Available: <https://www.youtube.com/watch?v=tU-iu02pYXo>
-
- [101] F. A. Storm, C. J. Buckley, and C. Mazzà, “Gait event detection in laboratory and real life settings: Accuracy of ankle and waist sensor based methods,” *Gait & Posture*, vol. 50, pp. 42-46, Oct. 2016, doi: 10.1016/j.gaitpost.2016.08.012.
-
- [102] T. Gujarathi and K. Bhole, “GAIT ANALYSIS USING IMU SENSOR,” in 2019 10th International Conference on Computing, Communication and Networking Technologies (ICCCNT), July 2019, pp. 1-5. doi: 10.1109/ICCCNT45670.2019.8944545.
-
- [103] K. L. Bacon et al., “Gait alterations associated with worsening knee pain and physical function: a machine-learning approach with wearable sensors in the Multicenter Osteoarthritis Study,” *Arthritis Care Res (Hoboken)*, vol. 76, no. 7, pp. 984-992, July 2024, doi: 10.1002/acr.25327.
-
- [104] J. Dissanayaka, “Flutter Bluetooth Communication,” Medium. Accessed: Sept. 25, 2025. [Online]. Available: <https://medium.com/@jdissanayaka11/flutter-bluetooth-communication-3f697ceda3b2>
-
- [105] “Feature Store for Real-Time ML,” Tecton. Accessed: Oct. 01, 2025. [Online]. Available: <https://www.tecton.ai/>
-
- [106] “Feast - The Open Source Feature Store for Machine Learning,” Feast. Accessed: Oct. 01, 2025. [Online]. Available: <https://feast.dev/>
-
- [107] “Event Sourcing: An Introduction,” Confluent. Accessed: Nov. 19, 2025. [Online]. Available: <https://www.confluent.io/learn/event-sourcing/>
-
- [108] “Ethics of Artificial Intelligence | UNESCO.” Accessed: Nov. 05, 2025. [Online]. Available: <https://www.unesco.org/en/artificial-intelligence/recommendation-ethics>
-
- [109] E. Shahabpoor and A. Pavic, “Estimation of vertical walking ground reaction force in real-life environments using single IMU sensor,” *Journal of Biomechanics*, vol. 79, pp. 181-190, Oct. 2018, doi: 10.1016/j.jbiomech.2018.08.015.
-
- [110] S. Krishnakumar, B.-J. F. van Beijnum, C. T. M. Baten, P. H. Veltink, and J. H. Buurke, “Estimation of Kinetics Using IMUs to Monitor and Aid in Clinical Decision-Making during ACL Rehabilitation: A Systematic Review,” *Sensors (Basel)*, vol. 24, no. 7, p. 2163, Mar. 2024, doi: 10.3390/s24072163.
-

-
- [111] X. Jiang, C. Napier, B. Hannigan, J. J. Eng, and C. Menon, "Estimating Vertical Ground Reaction Force during Walking Using a Single Inertial Sensor," *Sensors*, vol. 20, no. 15, p. 4345, Aug. 2020, doi: 10.3390/s20154345.
 - [112] DSwithBappy, End to end Deep Learning Project Implementation using MLOps Tool MLflow & DVC with CICD Deployment, (Nov. 24, 2023). Accessed: Oct. 01, 2025. [Online Video]. Available: <https://www.youtube.com/watch?v=NOIWzjJK-4>
 - [113] "Elyra Documentation — Elyra 4.0.1.dev0 documentation." Accessed: Oct. 01, 2025. [Online]. Available: <https://elyra.readthedocs.io/en/latest/>
 - [114] A. Rancea, I. Anghel, and T. Cioara, "Edge Computing in Healthcare: Innovations, Opportunities, and Challenges," *Future Internet*, vol. 16, no. 9, p. 329, Sept. 2024, doi: 10.3390/fi16090329.
 - [115] "Edge Computing | Accenture." Accessed: Nov. 05, 2025. [Online]. Available: <https://www.accenture.com/us-en/insights/cloud/edge-computing-index>
 - [116] "DVC/CML MLOps Tutorials," YouTube. Accessed: Sept. 30, 2025. [Online]. Available: <http://www.youtube.com/playlist?list=PL7WG7YrwYcnDBDuCkFbcyjnZQrdsKFbZ>
 - [117] "dvc.yaml Files," Data Version Control · DVC. Accessed: Nov. 28, 2025. [Online]. Available: <https://doc.dvc.org/user-guide/project-structure/dvcyaml-files>
 - [118] "Dossier Marktplaats - Movella DOT sensor."
 - [119] "Distributed Training and Experiment Tracking with Ray Train, MLflow, and MinIO," MinIO Blog. Accessed: Oct. 01, 2025. [Online]. Available: <https://blog.min.io/distributed-training-and-experiment-tracking-with-ray-train-mlflow-and-minio/>
 - [120] H. I. Fawaz and et al., "Deep Learning for Time Series Classification: A Review," *Data Mining and Knowledge Discovery*, 2019, doi: 10.1007/s10618-019-00619-1.
 - [121] Z. Zamanzadeh Darban, G. I. Webb, S. Pan, C. Aggarwal, and M. Salehi, "Deep Learning for Time Series Anomaly Detection: A Survey," *ACM Comput. Surv.*, vol. 57, no. 1, p. 15:1-15:42, Oct. 2024, doi: 10.1145/3691338.
 - [122] A. P. Pandelu, "Day 66: Data Versioning and Management — DVC, MLflow," Medium. Accessed: Nov. 24, 2025. [Online]. Available: <https://medium.com/@bhatadithya54764118/day-66-data-versioning-and-management-dvc-mlflow-2d53fe3232ee>
 - [123] SAI Conference Authors, "DataOps lifecycle with a case study in healthcare," 2022. [Online]. Available: https://thesai.org/Downloads/Volume14No1/Paper_15-DataOps_Lifecycle_with_a_Case_Study_in_Healthcare.pdf
 - [124] "Data Version Control · DVC," Data Version Control · DVC. Accessed: Oct. 01, 2025. [Online]. Available: <https://dvc.org/>
 - [125] "Data Quality with Dagster + GX." Accessed: Nov. 19, 2025. [Online]. Available: <https://dagster.io/blog/ensuring-data-quality-with-dagster-and-great-expectations>
 - [126] "Data Pipeline Architecture: 5 Design Patterns with Examples | Dagster Guides." Accessed: Nov. 19, 2025. [Online]. Available: <https://dagster.io/guides/data-pipeline-architecture-5-design-patterns-with-examples>
 - [127] "Data Pipeline Architecture 2024: Scalable, Secure & Reliable." Accessed: Oct. 06, 2025. [Online]. Available: <https://atlan.com/data-pipeline-architecture/>
 - [128] "Data Orchestration Tools: 10 Key Features & 10 Platforms to Know | Dagster Guides." Accessed: Nov. 19, 2025. [Online]. Available: <https://dagster.io/guides/data-orchestration-tools-10-key-features-10-platforms-to-know>
-

-
- [129] “Data Authenticity and Integrity in MinIO,” MinIO Blog. Accessed: Nov. 21, 2025. [Online]. Available: <https://blog.min.io/data-authenticity-integrity/>
-
- [130] “Dagster Supports Every Dev Lifecycle Step.” Accessed: Nov. 19, 2025. [Online]. Available: <https://dagster.io/blog/dagster-five-stages-development-lifecycle>
-
- [131] confluentinc/confluent-kafka-python. (Oct. 19, 2025). Python. Confluent Inc. Accessed: Oct. 20, 2025. [Online]. Available: <https://github.com/confluentinc/confluent-kafka-python>
-
- [132] “confluent_kafka API — confluent-kafka 2.12.0 documentation.” Accessed: Oct. 20, 2025. [Online]. Available: <https://docs.confluent.io/platform/current/clients/confluent-kafka-python/html/index.html>
-
- [133] “Confluent Schema Registry: Enforcing Data Contracts in Kafka,” Confluent. Accessed: Nov. 05, 2025. [Online]. Available: <https://developer.confluent.io/courses/apache-kafka/schema-registry/>
-
- [134] M. Popat, “Comparing the Total Cost of Ownership (TCO) of Cloud Storage vs. On-Premise Storage,” Medium. Accessed: Oct. 08, 2025. [Online]. Available: <https://mihirpopat.medium.com/comparing-the-total-cost-of-ownership-tco-of-cloud-storage-vs-on-premise-storage-78a0c602611c>
-
- [135] “CML · Continuous Machine Learning,” CML · Continuous Machine Learning. Accessed: Oct. 01, 2025. [Online]. Available: <https://cml.dev/>
-
- [136] S. Canali, V. Schiaffonati, and A. Aliverti, “Challenges and recommendations for wearable devices in digital health: Data quality, interoperability, health equity, fairness,” PLOS Digital Health, vol. 1, no. 10, p. e0000104, Oct. 2022, doi: 10.1371/journal.pdig.0000104.
-
- [137] “Canary Rollout Strategy | KServe.” Accessed: Nov. 24, 2025. [Online]. Available: <https://kserve.github.io/website/docs/model-serving/predictive-inference/rollout-strategies/canary>
-
- [138] F. Ferraris, I. Gorini, U. Grimaldi, and M. Parvis, “Calibration of three-axial rate gyros without angular velocity standards,” Sensors and Actuators A: Physical, vol. 42, no. 1, pp. 446-449, Apr. 1994, doi: 10.1016/0924-4247(94)80031-6.
-
- [139] “C4 Model,” C4 model. Accessed: Oct. 27, 2025. [Online]. Available: <https://c4model.com/>
-
- [140] ClickHouse, “Building StockHouse: Real-time market analytics with ClickHouse,” ClickHouse. Accessed: Nov. 28, 2025. [Online]. Available: <https://clickhouse.com/blog/building-stockhouse>
-
- [141] R. Maheshwar, “Building Reproducible ML Pipelines with DVC and GitLab,” Medium. Accessed: Nov. 24, 2025. [Online]. Available: <https://medium.com/@maheshwar.ramkrushna/building-reproducible-ml-pipelines-with-dvc-and-gitlab-0ab768eadb89>
-
- [142] “Building machine learning pipelines with Dagster | Dagster Docs.” Accessed: Nov. 03, 2025. [Online]. Available: <https://docs.dagster.io/guides/build/ml-pipelines/ml-pipeline>
-
- [143] “Build an ML app pipeline with GitLab Model Registry using MLflow,” about.gitlab.com. Accessed: Nov. 24, 2025. [Online]. Available: <https://about.gitlab.com/blog/build-an-ml-app-pipeline-with-gitlab-model-registry-using-mlflow/>
-
- [144] S. Das, “Best Practices for Dealing With Concept Drift,” neptune.ai. Accessed: Nov. 05, 2025. [Online]. Available: <https://neptune.ai/blog/concept-drift-best-practices>
-
- [145] “Backend Stores | MLflow.” Accessed: Nov. 24, 2025. [Online]. Available: <https://mlflow.org/docs/latest/self-hosting/architecture/backend-store/>
-

-
- [146] “Artifact Stores | MLflow.” Accessed: Nov. 24, 2025. [Online]. Available: <https://mlflow.org/docs/latest/self-hosting/architecture/artifact-store/>
-
- [147] Google Cloud, “Architecture for MLOps using TensorFlow Extended, Vertex AI Pipelines, and Cloud Build,” Google Cloud, 2024. [Online]. Available: <https://cloud.google.com/architecture/architecture-for-mlops-using-tfx-kubeflow-pipelines-and-cloud-build>
-
- [148] S. J. Warnett and U. Zdun, “Architectural Design Decisions for the Machine Learning Workflow,” *Computer*, vol. 55, no. 3, pp. 40-51, Mar. 2022, doi: 10.1109/MC.2021.3134800.
-
- [149] P. Kruchten, “Architectural Blueprints—The ‘4+1’ View Model of Software Architecture”.
-
- [150] “Apache Kafka,” Apache Kafka. Accessed: Oct. 01, 2025. [Online]. Available: <https://kafka.apache.org/>
-
- [151] “Apache Airflow,” Apache Airflow. Accessed: Oct. 01, 2025. [Online]. Available: <https://airflow.apache.org/>
-
- [152] C. Cramer, “Anterior Cruciate Ligament Treatment | Physi.nl,” Physi Tilburg. Accessed: Oct. 29, 2025. [Online]. Available: <https://www.fysi.nl/en/blog/anterior-cruciate-ligament-treatment/>
-
- [153] “Announcing KServe v0.11 | KServe.” Accessed: Nov. 24, 2025. [Online]. Available: <https://kserve.github.io/website/blog/kserve-0.11-release>
-
- [154] S. Bakker, “An MLOps Architecture for a Data Platform to Measure Personal Stress with Wearables.”
-
- [155] X. Liu, X. Zhang, B. Zhang, B. Zhou, Z. He, and T. Liu, “An IMU-Based Ground Reaction Force Estimation Method and Its Application in Walking Balance Assessment,” *IEEE Trans. Neural Syst. Rehabil. Eng.*, vol. 32, pp. 223–232, 2024, doi: 10.1109/TNSRE.2023.3347729.
-
- [156] D. Kovačević, “An illustrated guide to medical device software development,” Medical Device HQ. Accessed: Oct. 08, 2025. [Online]. Available: <https://medicaldevicehq.com/articles/the-illustrated-guide-to-medical-device-software-development-and-iec-62304/>
-
- [157] Z. Li and X. Zhang, “An Enhanced Autoencoder-based Anomaly Detection Model for time series Data from Wearable Medical Devices,” *IEEE Journal of Biomedical and Health Informatics*, pp. 1-13, 2024, doi: 10.1109/JBHI.2024.3434420.
-
- [158] S. Bakker, “An Architectural Framework for Stress Wearable Data Platforms.”
-
- [159] P. Heck, J. Snoeren, M. Veracx, and M. Peeters, “An Approach for Integrated Development of an MLOps Architecture”.
-
- [160] “Alibi Outlier/Drift Detection | KServe.” Accessed: Nov. 24, 2025. [Online]. Available: <https://kserve.github.io/website/docs/model-serving/predictive-inference/detect/alibi/alibi-detect>
-
- [161] “AI/ML in Kubernetes Best Practices: The Essentials | Wiz,” wiz.io. Accessed: Nov. 19, 2025. [Online]. Available: <https://www.wiz.io/academy/ai-ml-kubernetes-best-practices>
-
- [162] E. Anwarie, “ACL Injury Risk Assessment Using IMU Wearables”.
-
- [163] O. Jouini, K. Sethom, A. Namoun, N. Aljohani, M. H. Alanazi, and M. N. Alanazi, “A Survey of Machine Learning in Edge Computing: Techniques, Frameworks, Applications, Issues, and Research Directions,” *Technologies*, vol. 12, no. 6, p. 81, June 2024, doi: 10.3390/technologies12060081.
-
- [164] F. Wang, Y. Jiang, R. Zhang, A. Wei, J. Xie, and X. Pang, “A Survey of Deep Anomaly Detection in Multivariate Time Series: Taxonomy, Applications, and Directions,” *Sensors*, vol. 25, no. 1, p. 190, Jan. 2025, doi: 10.3390/s25010190.
-

-
- [165] T. Tan et al., “A scoping review of portable sensing for out-of-lab anterior cruciate ligament injury prevention and rehabilitation,” *npj Digit. Med.*, vol. 6, no. 1, p. 46, Mar. 2023, doi: 10.1038/s41746-023-00782-2.
 - [166] “A Reference Architecture for Governance of Cloud Native Applications.” Accessed: Nov. 19, 2025. [Online]. Available: <https://arxiv.org/html/2302.11617v2>
 - [167] D. Anguita, A. Ghio, L. Oneto, X. Parra, and J. L. Reyes-Ortiz, “A Public Domain Dataset for Human Activity Recognition using Smart-phones,” presented at the The European Symposium on Artificial Neural Networks, 2013. Accessed: Nov. 12, 2025. [Online]. Available: <https://www.semanticscholar.org/paper/A-Public-Domain-Dataset-for-Human-Activity-using-Anguita-Ghio/83de43bc849ad3d9579ccf540e6fe566ef90a58e>
 - [168] P. Ramasamy, P. Palani, G. Renganathan, K. Shimatani, A. Thondiyath, and Y. Kurita, “A Preliminary Data-Driven Approach for Classifying Knee Instability During Subject-Specific Exercise-Based Game with Squat Motions,” *Sensors*, vol. 25, no. 19, p. 6074, Jan. 2025, doi: 10.3390/s25196074.
 - [169] J. Anthony, “A Comprehensive Guide to Scaling Microservices,” *CrossAsyst*. Accessed: Nov. 19, 2025. [Online]. Available: <https://crossasyst.com/blog/scaling-microservices/>
 - [170] G. Regunath, “A Beginner’s Guide To Understanding Feature Stores,” *Advancing Analytics*. Accessed: Oct. 01, 2025. [Online]. Available: <https://medium.com/advancing-analytics/a-beginners-guide-to-understanding-feature-stores-df98decaa19a>
 - [171] “[2310.20071] FOCAL: Contrastive Learning for Multimodal Time-Series Sensing Signals in Factorized Orthogonal Latent Space.” Accessed: Sept. 25, 2025. [Online]. Available: <https://arxiv.org/abs/2310.20071>
 - [172] “6 Benefits of a Modern Data Pipeline & How to Build One | Dagster.” Accessed: Nov. 05, 2025. [Online]. Available: <https://dagster.io/learn/data-pipeline>
 - [173] “6 Benefits of a Modern Data Pipeline & How to Build One | Dagster.” Accessed: Nov. 05, 2025. [Online]. Available: <https://dagster.io/learn/data-pipeline>
 - [174] “3.1 - Introduction to IMUs and preprocessing.” Accessed: Sept. 26, 2025. [Online]. Available: <https://kaggle.com/code/widehealth/3-1-introduction-to-imus-and-preprocessing>
 - [175] “ML Done Right: Versioning Datasets and Models with DVC & MLflow,” *DEV Community*. Accessed: Nov. 05, 2025. [Online]. Available: <https://dev.to/aws-builders/ml-done-right-versioning-datasets-and-models-with-dvc-mlflow-4p3f>
-

Table 1: Overview of literature and online resources on DataOps & MLOps, data engineering workflows, and wearable sensor-based gait and knee instability analysis

B Architectural Principles

Based on the research, common architectural principles were extracted and used as requirements that the proposed data platform needs satisfy. These principles were then prioritized using the MoSCoW method, together with the stakeholders, to focus on the most critical aspects of the architecture. The architectural principles span four domains: 1) Data Management, 2) ML Lifecycle, 3) Infrastructure, and 4) Quality Attributes.

Data Management is critical for the proposed architecture. Table 1 defines architectural principles that specify how the architecture should handle data throughout its lifecycle.

ID	Requirement	Implemented	Source
AP-DM-0	The system <i>MUST</i> allow for the versioning of datasets for reproducibility and lineage control.	YES	[45, 46]
AP-DM-1	The system <i>MUST</i> enforce a unified and invariant data schema across components.	YES	[40]
AP-DM-2	The system <i>MUST</i> ensure loose data coupling by preferring data streams over direct APIs and support independent evolution of components.	YES	[40]
AP-DM-3	The system <i>MUST</i> store current and historical data for monitoring, diagnosis and retraining.	YES	[40]
AP-DM-4	The system <i>MUST</i> normalize data sources and remove invalid or noisy artifacts before training.	YES	[44, 45]
AP-DM-5	The system <i>MUST</i> support multi-source ingestion by handling streaming, query-based, batch, and manual inputs.	YES	[33, 45]
AP-DM-6	The system <i>MUST</i> provide both low-latency real-time streams and offline/backfill batch pipelines.	YES	[54]
AP-DM-7	The system <i>SHOULD</i> ensure lineage and auditability by tracking data provenance, transformations, and metadata to meet compliance requirements.	YES	[33, 45]
AP-DM-8	The system <i>SHOULD</i> implement data-centric operations designed around the 5Vs: volume, velocity, variety, veracity, and value.	YES	[40, 45]

Table 2: Data management Principles.

Collectively, the architectural principles in Table 1 ensure that the architecture can scale and adapt while preserving data quality and governance. Versioning and provenance tracking support reproducibility and auditability of models and decisions. Multi-source ingestion and support for both streaming and batch pipelines provide flexibility in how data is acquired and processed. Normalization and noise removal safeguard the integrity of training and monitoring signals, while enforcing a unified data schema reduces integration overhead between components. Finally, by explicitly designing around the five Vs of data (volume, velocity, variety, veracity, and value), the architecture is prepared to operate reliably under realistic workload conditions and evolving business needs.

ML Lifecycle encompasses the processes required to reliably transition models from initial experimentation to stable production use and to maintain their performance over time. Table 2 defines architectural principles that specify how the architecture should manage models across stages such as training, validation, deployment, and ongoing maintenance.

ID	Requirement	Implemented	Source
AP-ML-0	The system <i>MUST</i> provide a model registry with uniquely trackable versions and must support rollback to prior versions.	YES	[44, 46]
AP-ML-1	The system <i>MUST</i> automate data preparation and model training to enable repeatable, reliable workflows.	YES	[44, 46]
AP-ML-2	The system <i>MUST</i> validate models before release using holdout datasets and metric checks as gates.	YES	[44, 46]
AP-ML-3	The system <i>COULD</i> automatically deploy only validated models into serving environments.	YES	[44, 46]
AP-ML-4	The system <i>COULD</i> support scheduled and event-based retraining, including triggers from new data and drift alerts.	NO	[44, 46]
AP-ML-5	The system <i>COULD</i> monitor live model performance and drift and must trigger improvement workflows when thresholds are breached.	NO	[44, 46]
AP-ML-6	The system <i>COULD</i> support both batch and on-line/personalized learning for incremental model updates.	NO	[54]
AP-DM-7	The system <i>COULD</i> track experiments, including datasets, parameters, metrics, and associated metadata.	NO	[33, 46]

Table 3: ML Lifecycle Principles.

Collectively, the architectural principles in Table 2 extend the ML lifecycle with mechanisms for continuous adaptation and control. Automated deployment of validated models reduces operational friction and helps ensure that only vetted artifacts reach serving environments. While not all of these are implemented in the current architecture, they define a clear roadmap for evolving the ML platform towards more adaptive, observable, and experiment-driven operation.

Infrastructure is a foundational concern for the proposed platform because it determines how Data and ML related components are deployed, coordinated, and evolved over time. Table 3 outlines the infrastructure architectural principles that govern orchestration, CI/CD, deployment strategies, and runtime behavior across services.

ID	Requirement	Implemented	Source
AP-I-0	The system <i>MUST</i> coordinate pipelines and components (e.g., MLFlow, Kubeflow, Dagster, Kubernetes) through a unified orchestration layer.	YES	[44, 46]
AP-I-1	The system <i>MUST</i> provide CI/CD for ML, automatically building, testing, and deploying on code, data, and configuration changes.	YES	[44, 46]
AP-I-2	The system <i>COULD</i> enforce testing, including data quality checks and unit tests, and must use a staging environment before production runs.	NO	[44, 45]
AP-I-3	The system <i>COULD</i> stage deployments and validate changes in staging before promoting to production.	NO	[44, 45]
AP-I-4	The system <i>SHOULD</i> support edge and local-first processing to reduce latency and enhance privacy.	YES	[40]
AP-I-5	The system <i>COULD</i> shard and replicate data and services to achieve fault-tolerance and horizontal scalability.	NO	[40]
AP-I-6	The system <i>MUST</i> use asynchronous pub/sub messaging to decouple components.	YES	[40]
AP-I-7	The system <i>MUST</i> allow services to autonomously join and leave and must support adding new data sources at runtime without downtime.	YES	[40]
AP-I-8	The system <i>COULD</i> optimize compute and storage usage and must support deployment on commodity and IoT-class devices where appropriate.	YES	[40]
AP-I-9	The system <i>MUST</i> apply DevOps practices (IaC, CI/CD, monitoring) consistently to both data and ML components.	YES	[33, 54]

Table 4: Infrastructure Principles.

Collectively, the architectural principles in Table 3 equip the architecture to support complex, distributed Data and ML workloads in a scalable and resilient manner. A unified orchestration layer, combined with CI/CD pipelines tailored to code, data, and configuration changes, enables reproducible and automated delivery across the stack. Asynchronous pub/sub messaging and the ability for services to autonomously join and leave the system support loose coupling and dynamic evolution of the architecture, including the addition of new data sources at runtime without downtime. The consistent application of DevOps practices (IaC, CI/CD, monitoring) to both Data and ML related components ensures operational coherence across the platform. While not all of these are implemented in the current architecture, they define a clear roadmap for evolving the infrastructure towards a more flexible, robust, and production-grade Data and ML platform.

Quality Attributes describe the cross-cutting, non-functional properties that determine the system’s trustworthiness, and robustness in practice. Table 4 summarizes the corresponding architectural principles, covering reproducibility, robustness to noise and drift, explainability, fairness, privacy and security, observability, and human oversight.

[40, 44, 33, 54, 45, 46] - [8], [9], [10], [11], [17], [18]

ID	Requirement	Implemented	Source
AP-QA-0	The system <i>MUST</i> ensure reproducibility, such that the same inputs produce the same outputs, and must version code, data, and models.	YES	[33, 54, 46]
AP-QA-1	The system <i>COULD</i> tolerate noise, anomalies, and drift and must handle adversarial or unexpected cases gracefully.	YES	[33, 54]
AP-QA-2	The system <i>SHOULD</i> provide model explainability, including feature importance, data lineage, and rationale for decisions where feasible.	NO	[33, 54]
AP-QA-3	The system <i>COULD</i> evaluate performance across relevant groups, must document limitations, and must include mechanisms to detect and mitigate bias.	NO	[33, 54]
AP-QA-4	The system <i>WOULD NOT</i> protect privacy and security by anonymizing data where appropriate, encrypting data in transit and at rest, enforcing access control, and complying with GDPR and other applicable regulations.	N/A	[40, 33]
AP-QA-5	The system <i>COULD</i> detect data/model drift and failures and must support autonomous retraining and scaling responses.	NO	[40, 45, 46]
AP-QA-6	The system <i>SHOULD</i> log data transformations, training runs, and model decisions to provide full traceability for audits and investigations.	YES	[33, 45, 46]
AP-QA-7	The system <i>MUST</i> provide expert dashboards and must support human interventions critical workflows.	YES	[33]

Table 5: Quality Attributes Principles.

Collectively, the architectural principles in Table 4 ensure that the system can deliver reliable and accountable Data and ML outcomes over time. Reproducibility through versioning of all artifacts underpins trustworthy experimentation and auditing, while robustness to noise, anomalies, and drift helps maintain performance as conditions evolve. While not all of these are currently implemented, they outline a clear roadmap for evolving the platform towards a more robust, transparent, and ethically grounded Data and ML platform.